what if the distance to the branch target address is outside the range we can represent with a 13-bit (signed) offset?

could use the “j” pseudoinstruction:

```
beq s1, s2, faraway ⇨ bne s1, s2, skip
                      j faraway
                      skip: …
```

the “j” pseudoinstruction is implemented using the jal (“jump and link”) instruction, which is also used for implementing procedures, as we’ll soon see

“jal” is the only instruction using the J-type format, a variant of the U-type format; with the J-type format we can construct a 21-bit offset (sign-extended to 32 bits); 20 of the bits come from the instruction, similarly as with branches a 0 bit is added at the right (so offset must be a multiple of two)

ordering of the bits in the immediate seems peculiar and is for low-level hardware optimization purposes (refer to Figs. 4.17, 4.18 in text) … note that we will never program directly in machine language, and it is easy for the assembler to handle the details of this strange bit ordering …

but what about a situation like this:

```
beq s1, s2, reallyreallyfaraway
```

there’s a jalr instruction we can use, where the target address is taken from a register; jalr has an important use in procedures

Procedures in RISC-V

(reference: text Section 2.8)

unlike with a high-level language, with assembly language programmer must explicitly handle many of the details of procedure call implementation

invoking / returning from a procedure

use jal instruction (e.g. "jal ra, myproc") to invoke a procedure

- "jal" does both a jump to the address given as an operand, and saves the return address (address of sequentially following instruction) in the specified register (usually ra)

can then use "jalr zero, 0(ra)" to return (what format do you think "jalr" uses?)

passing parameters / returning results

in RISC-V systems, by convention:

- registers a0 - a7 are used to pass parameters
- registers a0, a1 are used for return values

example: procedure to find largest element in array

```
        # outline of example main program
        # calls procedure twice for different arrays, then compares results
        .section .bss
A:      .space 40               # array of 10 words
B:      .space 80               # array of 20 words
        …
        .section .text
        …
        la a0, A                # array starting address passed in a0
        li a1, 10               # number of array elements passed in a1
        jal ra, max             # or could use pseudoinstruction “call max”
        mv s0, a0               # result returned in a0
        la a0, B
        li a1, 20
        jal ra, max
        bgt a0, s0, Bhasbiggest     # what if we’d used t0 instead of s0?
        …

        # procedure can be in same file, or if in separate file need
        # “.globl max” in that file, and need to link the .o file for
        # the procedure with the .o file for the main program

max:    lw t0, 0(a0)            # assume at least 1 element in array
loop:   addi a1, a1, -1
        beqz a1, done
        addi a0, a0, 4
        lw t1, 0(a0)
        bge t0, t1, loop
        mv t0, t1
        j loop
done:   mv a0, t0
        jalr zero, 0(ra)        # or could use pseudoinstruction “ret”
```

register usage convention

don't want procedure code to overwrite values in registers that are being used by the caller!

- could save all register contents in memory before calling procedure, and then restore these contents after return, but high overhead with this approach

in RISC-V, adopt register usage convention in which general-purpose registers divided into two groups:

- caller saved registers: t0 – t7 (x5-x7, x28-31)
- callee saved registers: s0 – s11 (x8, x9, x18-27)

but where exactly are register contents saved?

program stack

operating system allocates each process a block of memory for use as a stack (of memory words)

register sp used as "top of stack" pointer

stack grows "downwards" in memory

example code for pushing a word on stack:

```
addi sp, sp, -4
sw s0, 0(sp)
```

code for popping a word, or loading a word from inside stack?

stack can be used to support procedure calls:

- saving register contents
- passing additional parameters to a procedure (9^{th}, 10^{th}, ...)
- local data storage

procedure call frame

items that are put on the stack to support a procedure call are organized in a consistent way; structure termed a procedure call frame

RISC-V convention for a procedure call frame:

bottom: - 9^{th}, 10^{th}, ... parameters (if any)
- saved contents of argument registers (if any)
- saved return address
- saved contents of callee saved registers (if any)

top: - local data (if any)

with nested calls, recursive procedures, multiple call frames can be on the stack at once … in such cases will need to (at least) save contents of ra on stack!